

Failure Analysis Report

Project: PHI / PII De-Identification Gateway

Date: 23 August 2026

1. Purpose of the Report

This report explains the main problems found while testing the PHI / PII De-Identification Gateway.

The testing was done to check whether the system could identify and remove sensitive information from clinical notes. During the initial testing, a few cases were missed. These issues were analysed, fixed, and tested again.

The report covers the failures found during testing, the reason behind each failure, the changes made, and the final results.

2. Failure Analysis

2.1 Patient Names Were Not Detected in Some Cases

Problem

The first version of the system mainly looked for names that appeared after titles such as **Dr.** or **Mr.**

Because of this, names written without a title were sometimes missed.

For example:

- Allison Hill
- Valerie Gray

The system also missed names written as uppercase headers, such as:

WHITFIELD, MARCUS D.

Reason

The original Regex rules depended too much on prefixes such as `Dr .` and `Mr .`.

In real clinical notes, a patient's name can appear in different formats. It may be written directly in a sentence or appear as a heading without any title.

Fix

A new `NAME_BANNER` Regex was added to detect uppercase names used as headers.

A separate **bare-name heuristic** was also added to identify normal names that appear without titles. This was combined with the **spaCy NER** fallback.

After these changes, the leak rate went from **98.0% to 0.0%** across 50 clinical evaluation notes.

Status: Resolved and Verified

2.2 Transformer Analyzer Stopped Too Early

Problem

During the next round of testing, another issue was found in `ner_model.py`.

The transformer analyzer had an early return:

```
if spans: return spans
```

Once the transformer detected an entity, the function returned immediately. Because of this, the other scanners did not get a chance to check the remaining text.

This resulted in **49 out of 50 synthetic notes containing leaks** during that test.

Reason

The problem was caused by the early exit in the detection flow.

The system was supposed to use multiple detection methods, but the early return stopped the process after the transformer produced a result.

Fix

The early return was removed from `TransformerPHIAnalyzer.analyze()`.

After the change, the transformer, spaCy NER, and heuristic scanners could all run before the final result was produced.

The leak rate was then verified at **0.0%**.

Status: Resolved and Verified

2.3 Regex Replaced the Field Name Along With the Patient Name

Problem

An issue was also found while processing fields such as:

```
Patient: John Doe
```

The Regex logic could replace the complete match instead of only replacing `John Doe`.

This meant that the field name **Patient:** could also be affected.

Reason

The code was using:

```
m.group(0)
```

instead of:

```
m.group(1)
```

`group(0)` represents the complete match, while `group(1)` represents the captured value that needed to be replaced.

Fix

The Regex analyzer was changed to use the correct capture group.

This keeps the field label unchanged while replacing only the sensitive value.

For example:

```
Patient: John Doe
```

becomes:

```
Patient: [PSEUDONYM]
```

The field name remains intact.

Status: Resolved and Verified

3. Technical Trade-offs

3.1 Model Fine-Tuning

The system uses **BioClinical-ModernBERT-base** for PHI / PII detection.

The model has **149,633,317 parameters**, which is within the required limit of 1 billion parameters.

The model was fine-tuned using the **HuggingFace Trainer** on a 50k subset of `temlm-foundation/Technetium-I`.

A **3x weighted loss** was also used through `WeightedTrainer`.

The fine-tuning configuration was saved in:

```
models/adapters/adapters_config.json
```

The parameter count was checked and verified.

3.2 CPU Latency

The system was tested on an Intel CPU environment.

The measured processing time was:

- **P50:** 175.44 ms
- **P95:** 262.67 ms

P50 represents the typical processing time, while P95 shows the processing time for slower cases.

A GPU-based production setup using **TensorRT / ONNX** is estimated to achieve around **30 ms P50**.

This GPU number is only an estimate. The actual measured result in this evaluation is the CPU latency.

3.3 Utility Evaluation

The project also needed to check whether the clinical text remained useful after de-identification.

The planned evaluation used **google/flan-t5-small** with 10 QA pairs per note.

This complete QA test was not run because of CPU execution-time limitations.

Instead, the evaluation used a **word-overlap score** as a proxy for checking how much useful text was retained.

The recorded utility preservation score was:

99.21%

This score should be treated as a proxy metric and not as a direct clinical QA result.

4. Benchmark Results

Test Run	Approach	Leak Rate	P50	P95
Run 01 – Initial	Naive Prefix Regex	98.0%	172.11 ms	276.06 ms
Run 02 – Final	Regex + spaCy + Heuristic	0.0%	175.44 ms	262.67 ms

Run 01 – Initial Version

The initial version used mainly prefix-based Regex rules.

It had a **98.0% leak rate** because names without titles and uppercase header names were not detected properly.

Run 02 – Final Version

The final version uses three detection methods:

Regex + spaCy NER + Heuristic scanning

The final test recorded:

- **0.0% leak rate**
- **100% recall across 50 gold notes**
- **100% recall across 50 synthetic notes**
- **175.44 ms P50 CPU latency**
- **262.67 ms P95 CPU latency**
- **Zero leaks in the tested notes**

5. Conclusion

The initial version of the system had problems detecting patient names in different formats. Names without titles and uppercase header names were the main cases that were missed.

During further testing, an early return in the transformer analyzer was also found. This prevented the other detection methods from checking the remaining text. A Regex field-matching issue was identified as well.

These problems were fixed by adding better name detection, allowing all detection methods to run, and correcting the Regex capture group.

The final version uses **Regex, spaCy NER, and heuristic scanning** together. In the final evaluation, the system recorded a **0.0% leak rate** and **100% recall across the tested gold and synthetic notes**.

The CPU latency and utility results are also reported separately from estimated production performance, so the report reflects what was actually tested.